

bok25

**基
礎**

課、操作系統



过程调用vs系统调用



过程调用

```
#include <stdio.h>
int add(int x, int y);
int main() {
    int result;
    int a = 5;
    int b = 3;
    result = add(a, b);
    printf("Result: %d\n", result);

    return 0;
}

int add(int x, int y) {
    return x + y;
}
```

过程调用也称为函数调用或子程序调用
它允许程序代码被组织成块，每个块完成特定的任务，这些块可以在程序的多个地方调用



过程调用

过程P

调用者caller

过程Q

被调用者callee

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P



过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

被调用者callee

```
int add(int x, int y){  
    return x+y;  
}
```

调用者caller

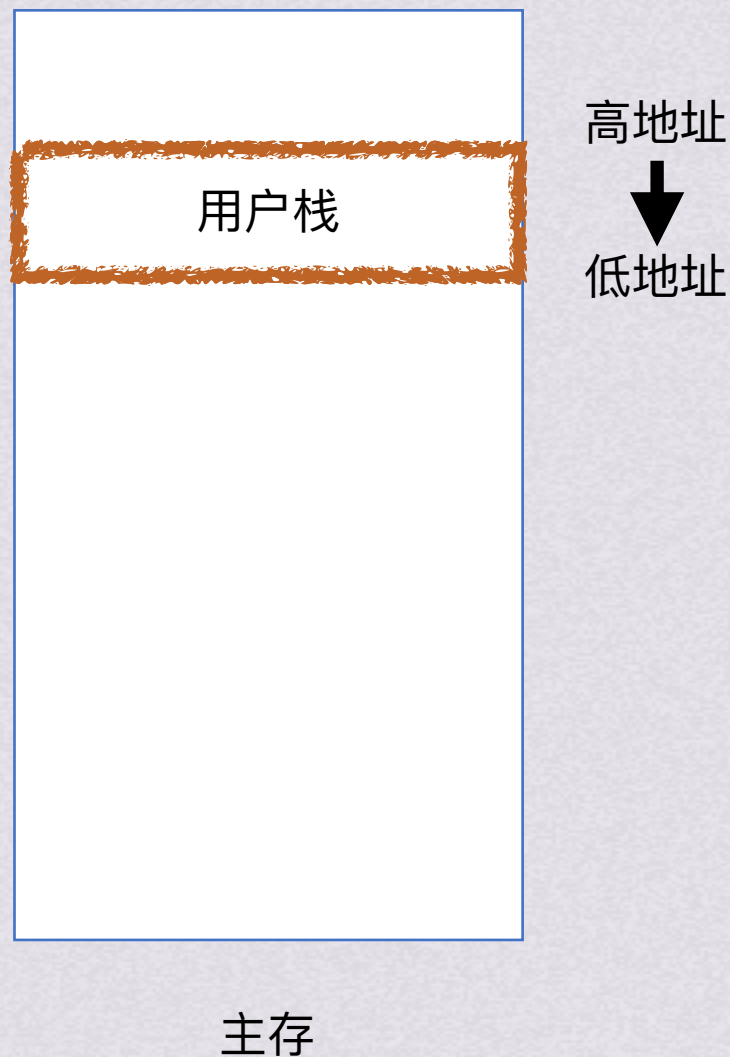
```
int caller()  
{  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```




过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```

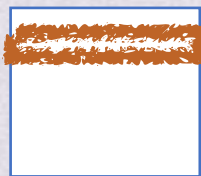




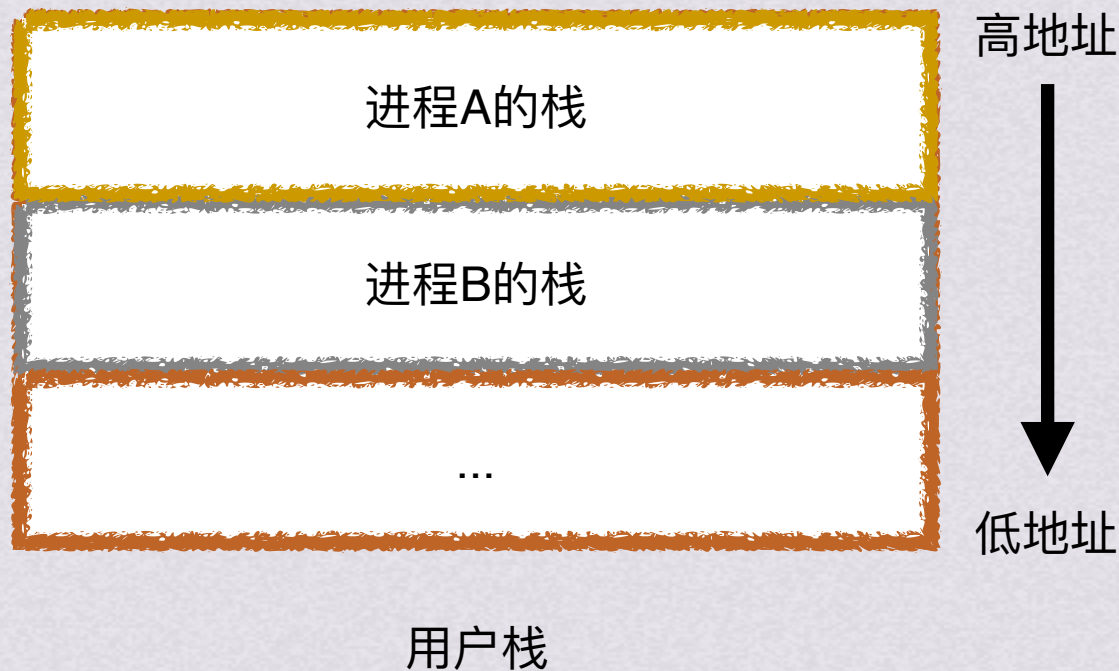
过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```



假设当前进程A和进程B正在使用主存栈区



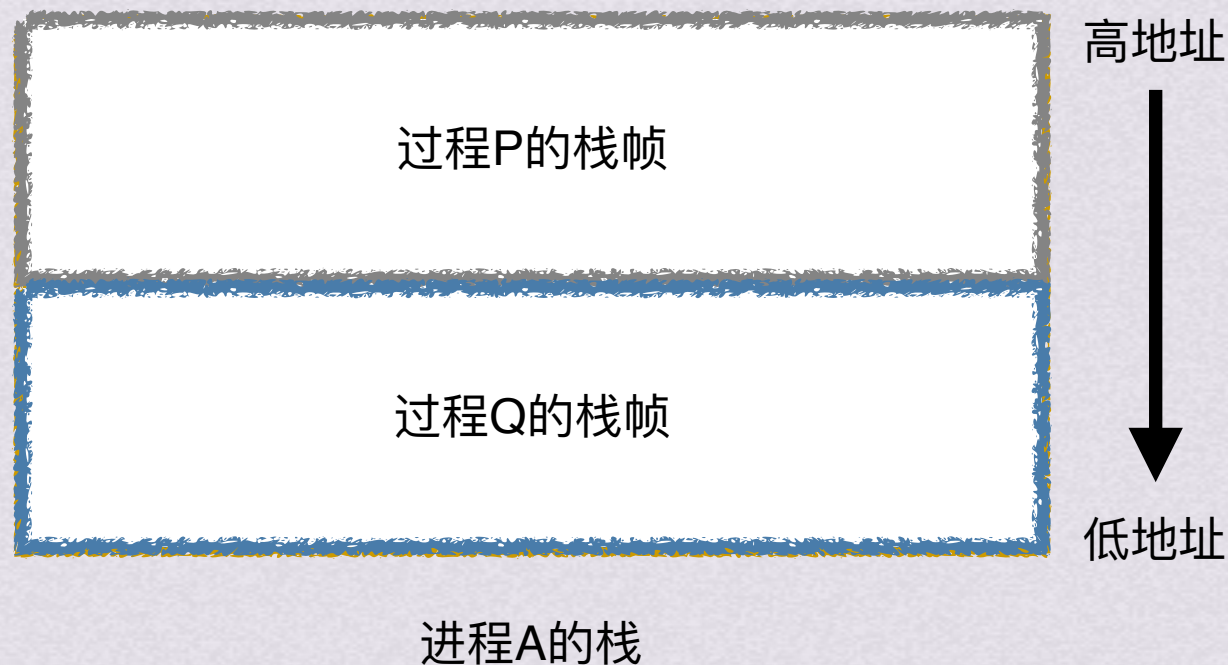


过程调用



- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

假设进程A中过程P正在调用过程Q



```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```




过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

假设调用caller()函数的过程是main()
caller()函数没有参数，所以直接保存main函数返回地址，控制转移到caller()

main()的栈帧 {

main函数返回地址

高地址



低地址

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```



过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

为即将调用的add()函数分配局部变量

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```



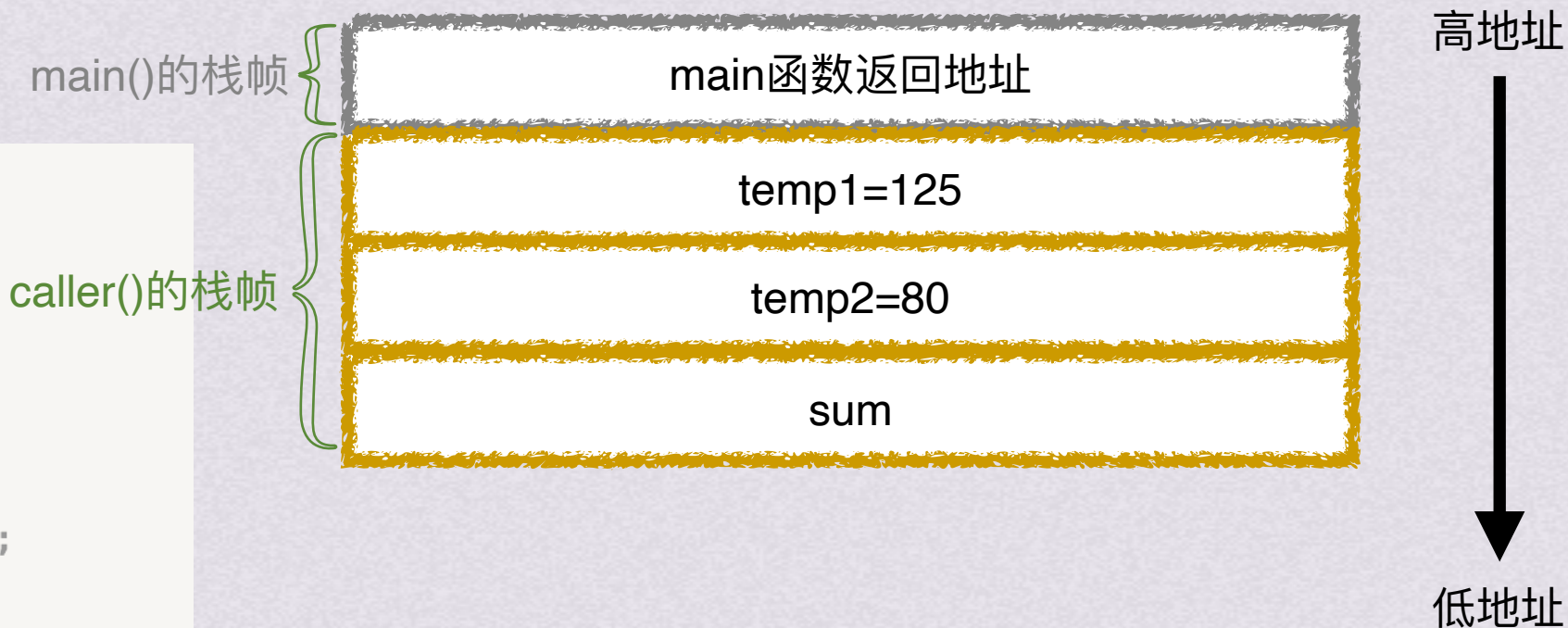


过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

初始化局部变量，将值复制给temp1和temp2

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```



尽管栈的基本操作限制于栈顶，但函数可以通过使用基指针ebp来访问其栈帧内的任意位置

mov [ebp-4], 125 ; 将 125 存储到 ebp-4 的位置，即 temp1 的位置

mov [ebp-8], 80 ; 将 80 存储到 ebp-8 的位置，即 temp2 的位置

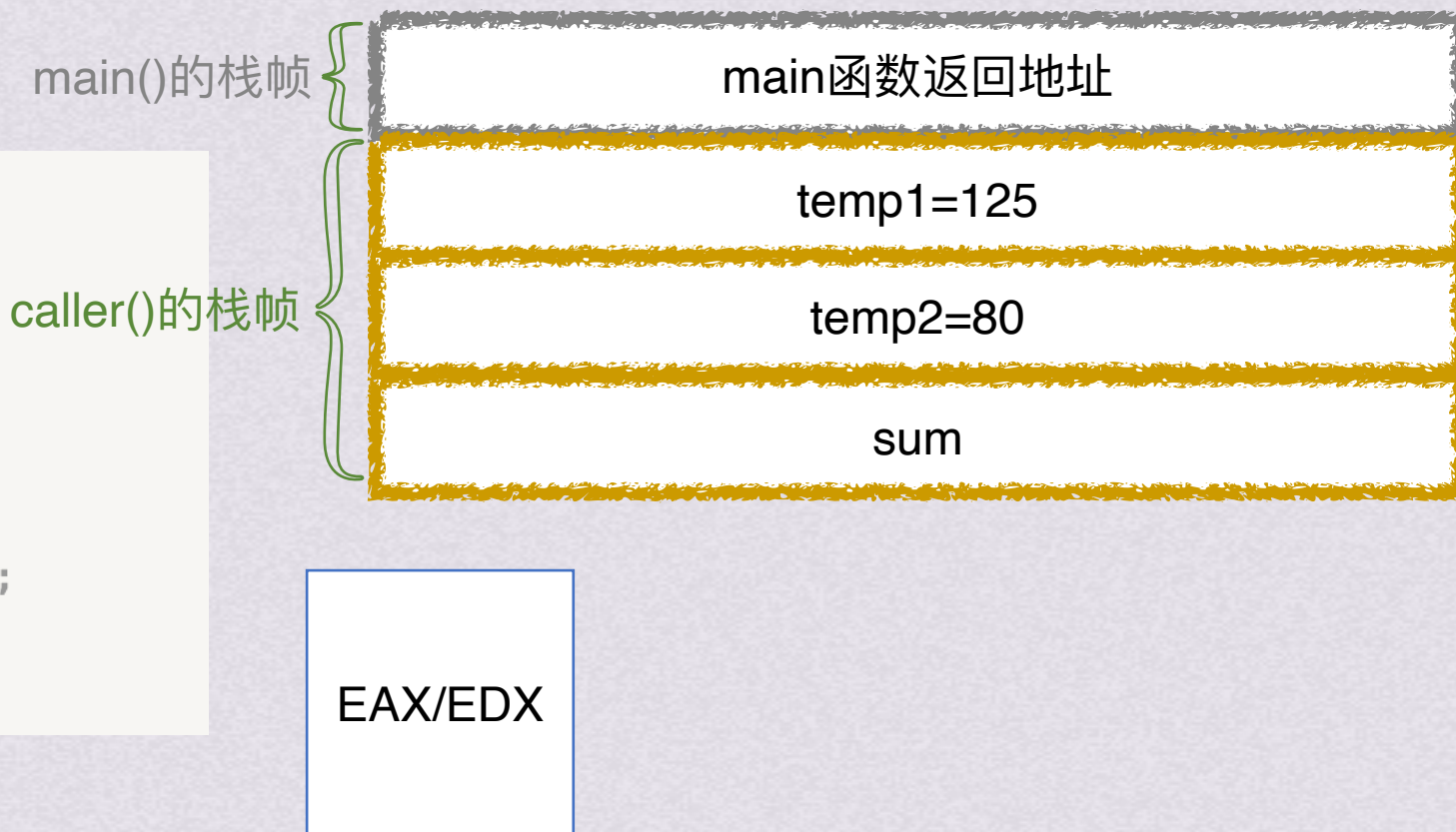


过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

将参数通过寄存器的中转再次压入栈的参数调用区

```
int add(int x, int y){  
    return x+y;  
}  
  
int caller(){  
    int temp1 = 125;  
    int temp2 = 80;  
    int sum = add(temp1, temp2);  
    return sum;  
}
```

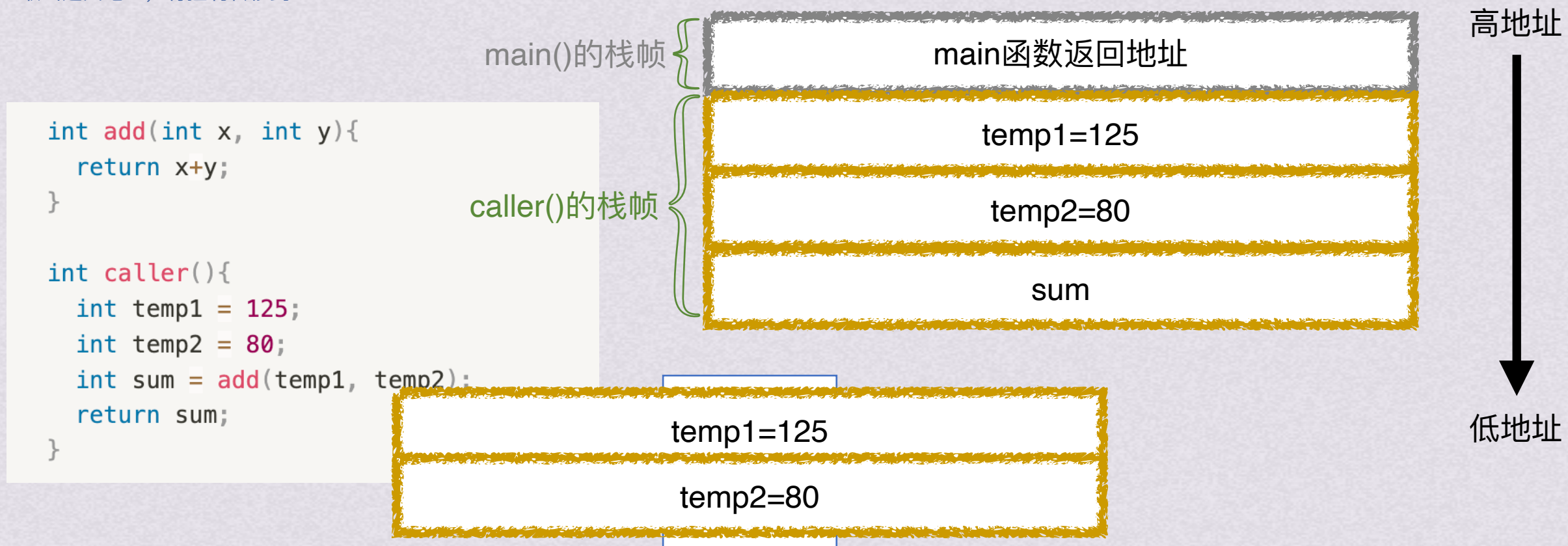




过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

将参数通过寄存器的中转再次压入栈的参数调用区



过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

将参数通过寄存器的中转再次压入栈的参数调用区

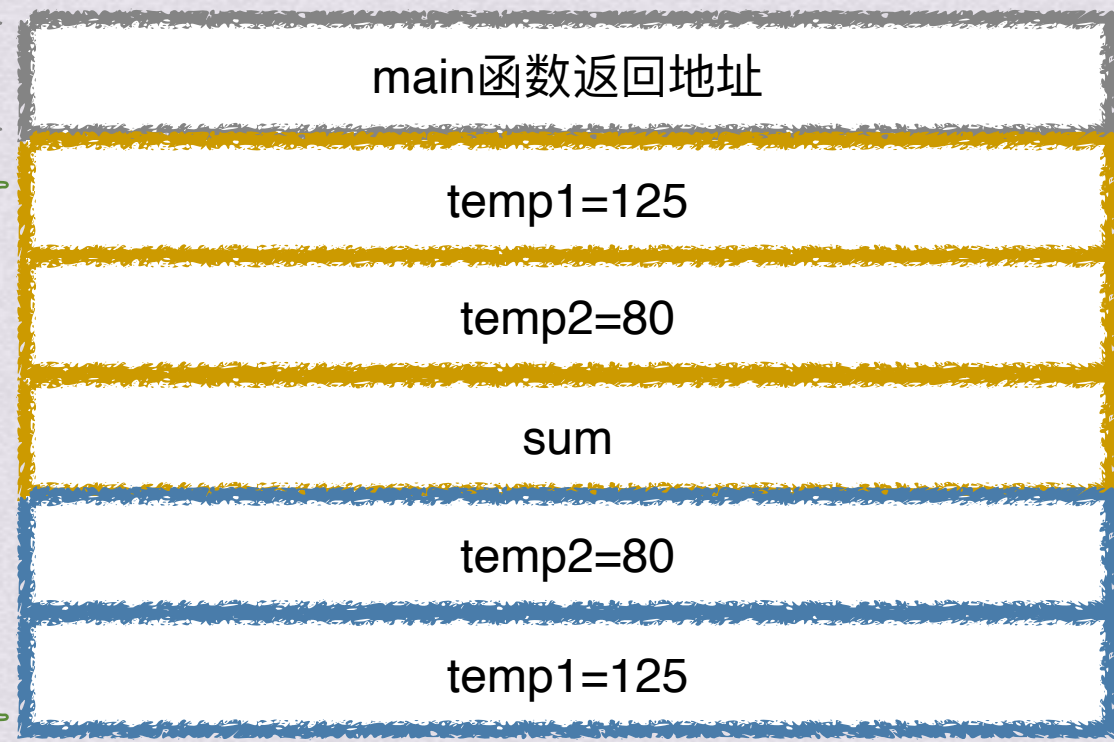
```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

main()的栈帧 {

caller()的栈帧 {

EAX/EDX



高地址
↓
低地址

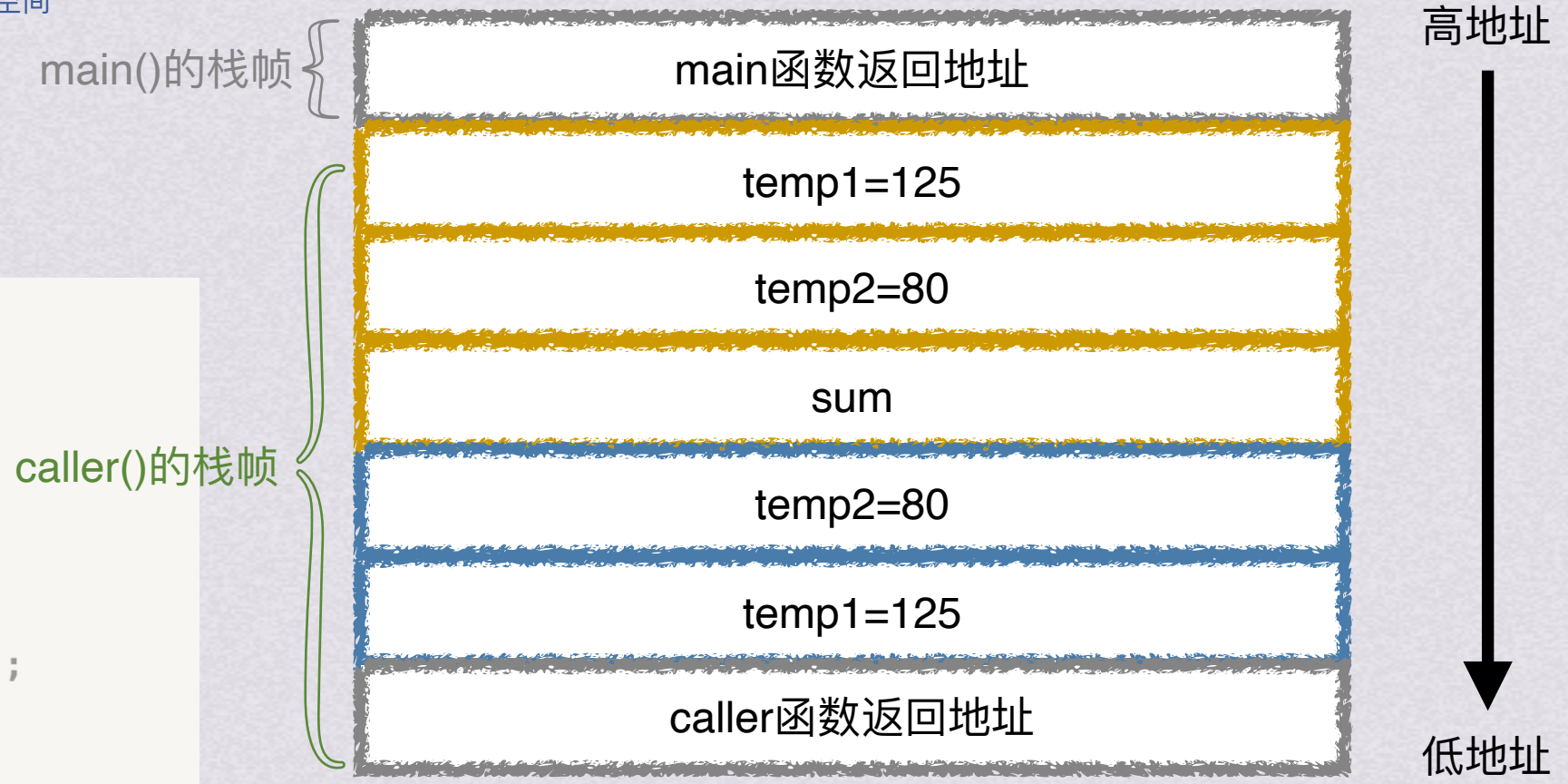
过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

执行call指令，调用add()函数
 将当前的PC值（即下一条指令的地址）压栈作为调用结束的返回地址，
 并将PC值设置为被调用函数的起始地址，从而将执行控制权转移给被调用函数

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```



过程调用

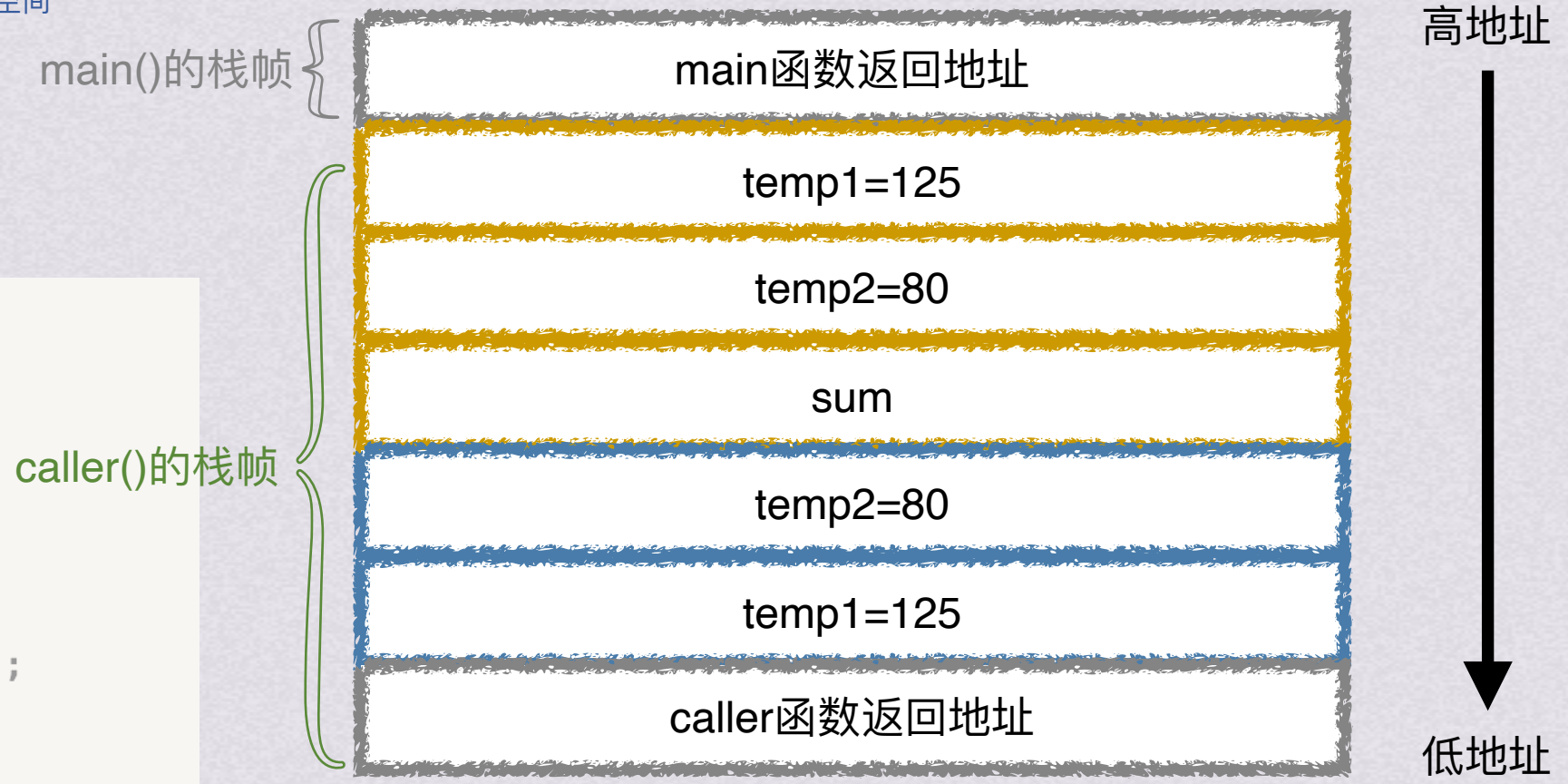
- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

add()的返回值被放在寄存器eax中，通过中转写入sum

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

205



过程调用

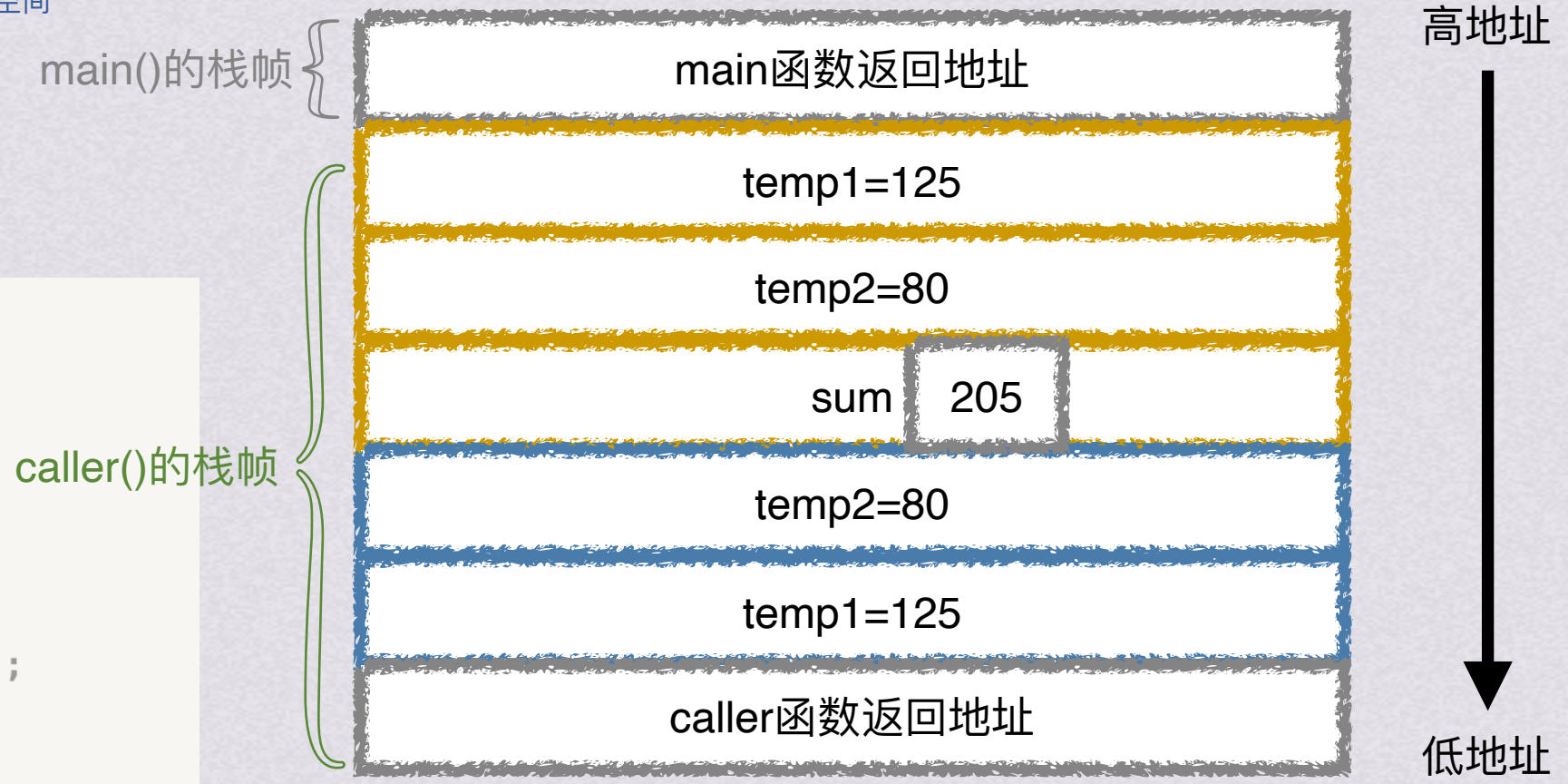
- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

add()的返回值被放在寄存器eax中，通过中转写入sum

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

EAX



过程调用

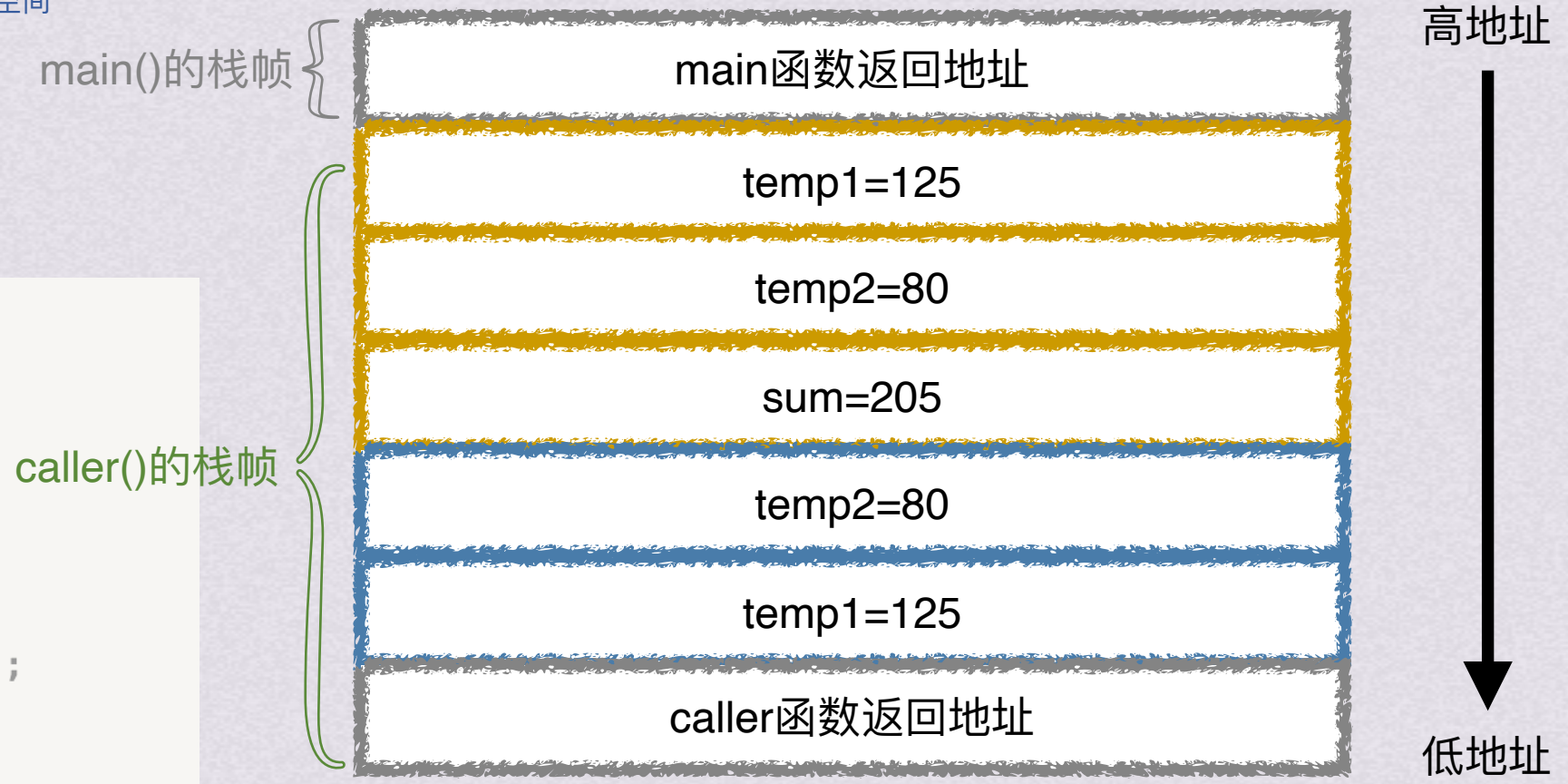
- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

add()的返回值被放在寄存器eax中，通过中转写入sum

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

EAX



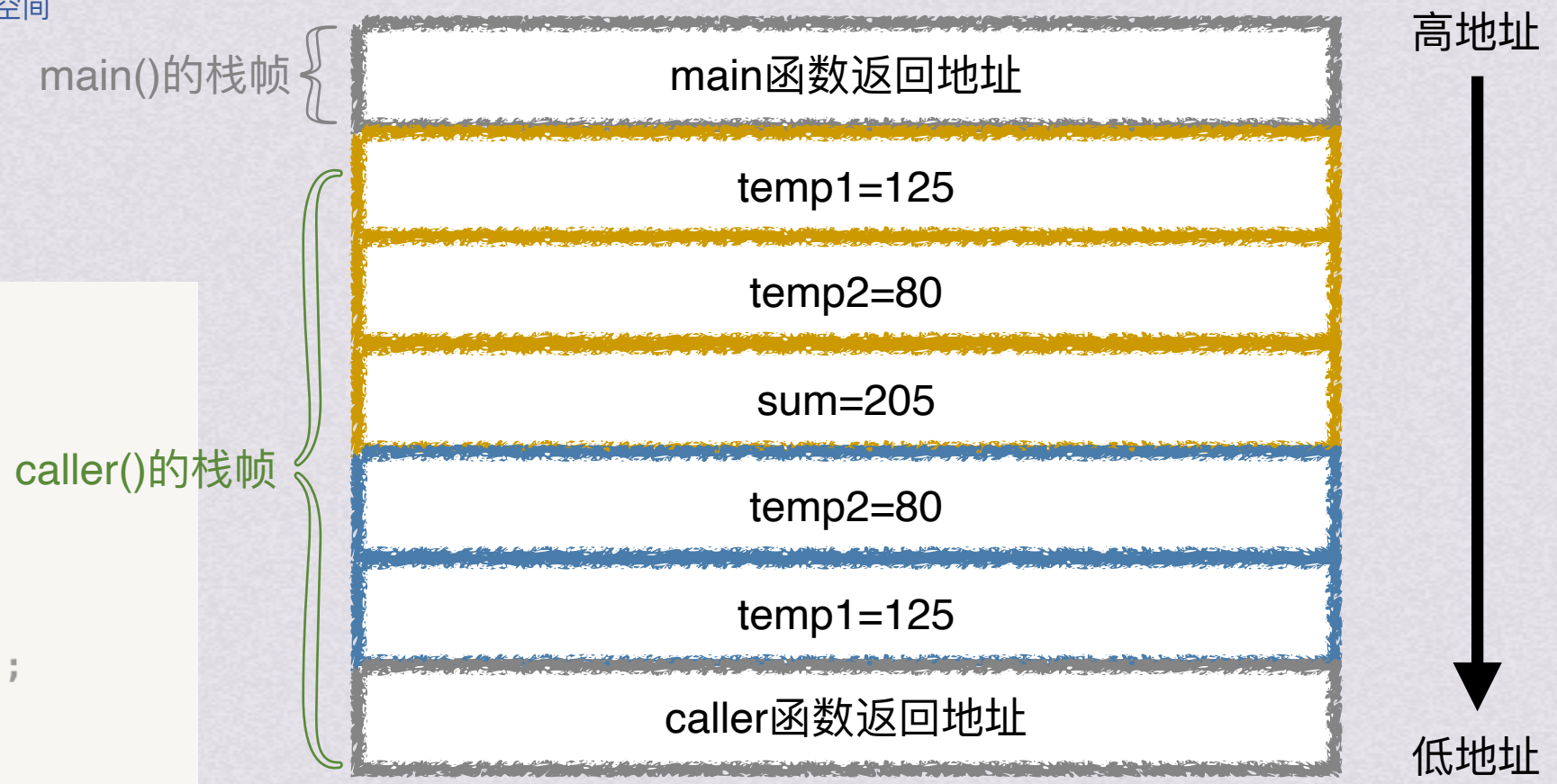
过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

add()函数执行ret指令
 caller函数返回地址出栈，修改PC值回到caller()
 add()栈帧被清理



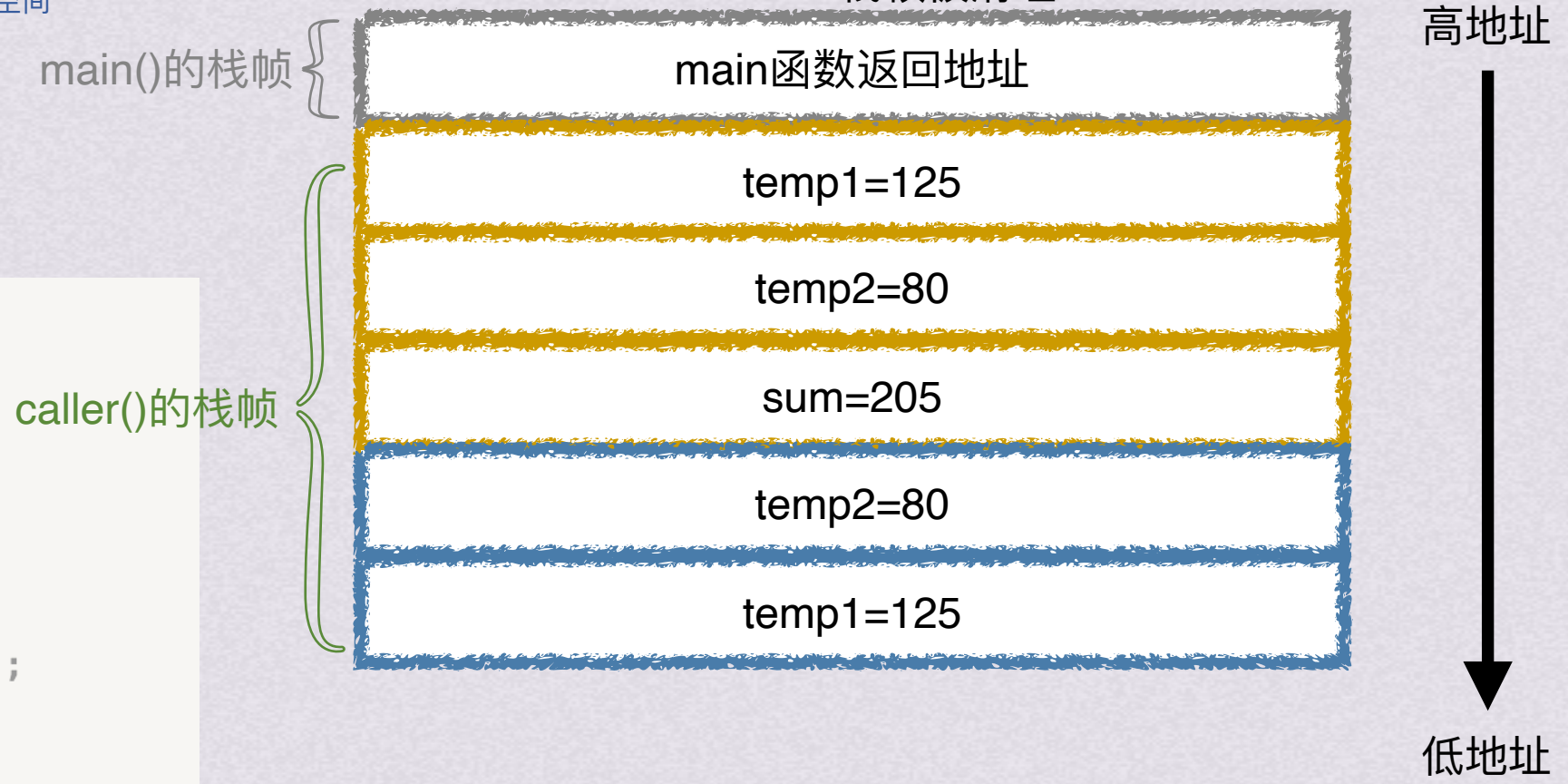
过程调用

- 1.P将入口参数放在Q能访问到的地方
- 2.P将返回地址存到特定的地方，然后将控制转移到Q
- 3.Q保存P的现场，并为自己的非静态局部变量分配空间
- 4.执行Q的过程体
- 5.Q恢复P的现场，并释放局部变量所占空间
- 6.Q取出返回地址，将控制转移到P

```
int add(int x, int y){
    return x+y;
}

int caller(){
    int temp1 = 125;
    int temp2 = 80;
    int sum = add(temp1, temp2);
    return sum;
}
```

caller()函数执行ret指令
Main函数返回地址出栈，修改PC值回到caller()
caller栈帧被清理

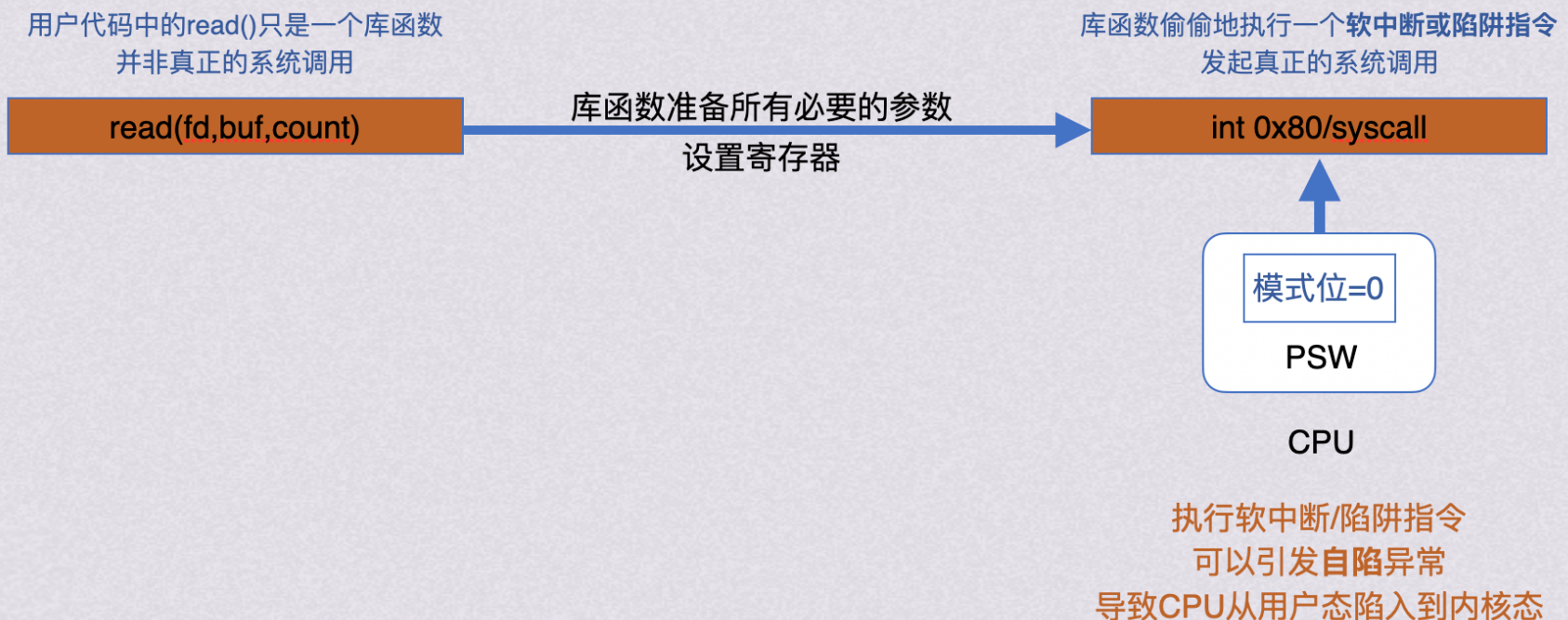




系统调用

系统调用

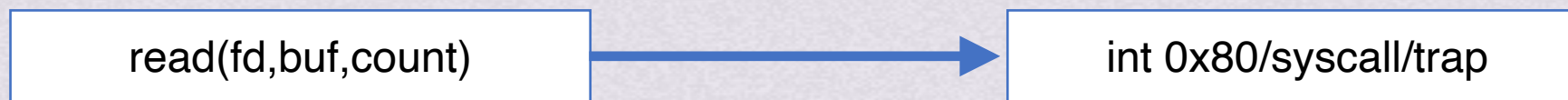
系统调用的过程



系统调用提供了用户程序和OS内核之间的接口
应用程序可以通过系统调用来获得OS的服务



系统调用



EAX



EAX



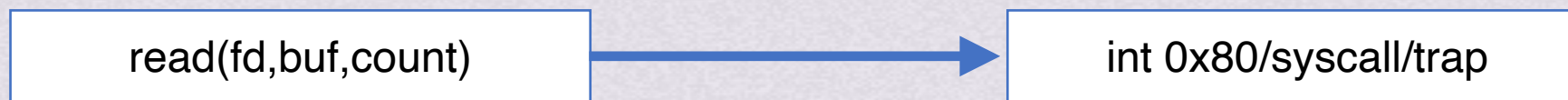
EAX



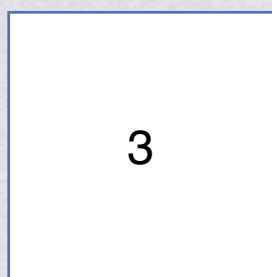
EAX



系统调用



将read对应的系统调用号3放入EAX



EAX



EBX



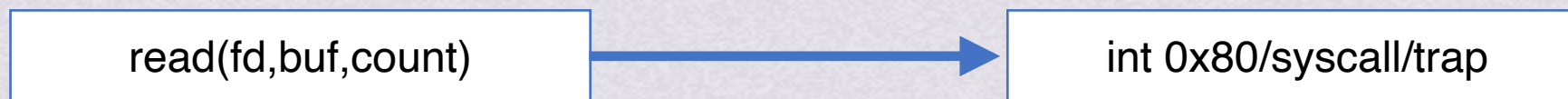
ECX



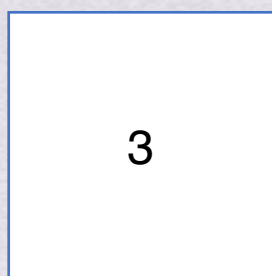
EDX



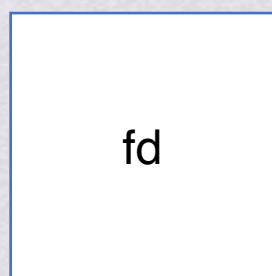
系统调用



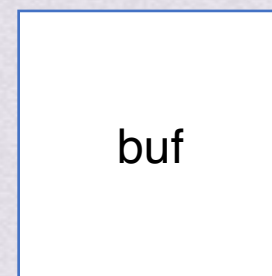
将参数分别放在EBX、ECX、EDX三个寄存器



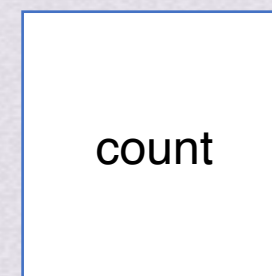
EAX



EBX



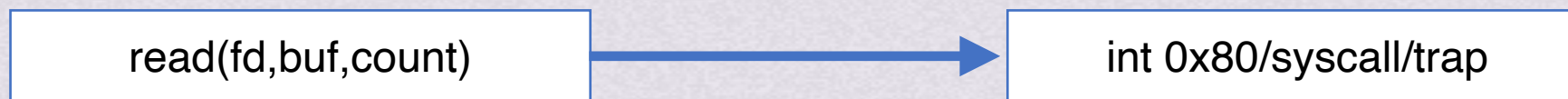
ECX



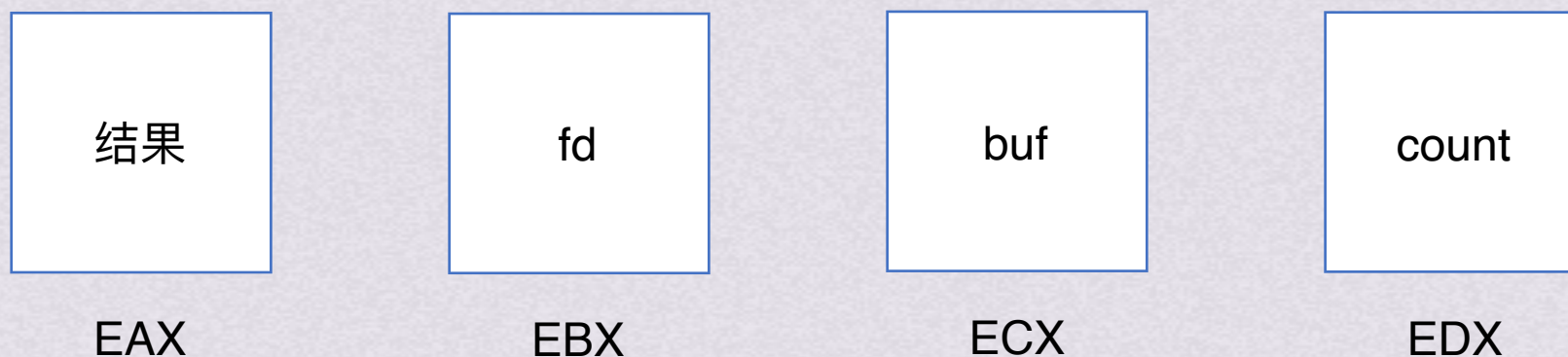
EDX



系统调用



CPU执行OS内核函数，实现读取文件，将结果存放在EAX





对比

过程调用没有引发中断，而系统调用是通过发起一个软中断实现调用OS内核代码的



对比

过程调用在调用时只需要保存PC而不需要保存PSW，
而系统调用需要正常地进行（内部）中断处理，因此需要保存PC和PSW



对比

过程调用全程CPU都处于同一个工作模式，即用户态或内核态，
而系统调用会导致CPU从用户态切换到内核态



对比

过程调用的被调用函数会出现在代码中，而系统调用通过调用一个库函数即可实现，OS内核的代码不会出现在程序代码中



对比

过程调用通过CALL或JMP指令进入
而系统调用通过INT或TRAP指令进入



对比

过程调用可以嵌套，而系统调用一般不允许发生嵌套或递归